

Câu 1: Hàm F được xây dựng bên dưới để cài đặt thuật toán gì?

```
void F(int a[], int n)
{
    for (int i = 1; i < n; i++)
    {
        int item = a[i];
        int j;
        for (j = i - 1; j >= 0; j--)
        {
            if (a[j] < item)
                break;
            a[j + 1] = a[j];
        }
        a[j + 1] = item;
    }
}
```

Thuật toán: Insertion Sort
Thứ tự sắp xếp: tăng dần
Hoạt động tốt với mảng nhỏ hoặc gần như đã sắp xếp.
Độ phức tạp:
Tốt nhất: $O(n)$
Xấu nhất: $O(n^2)$

Câu 2: Quan sát dãy giá trị sau:

84.2	68.4	59.2	49.7	31.8	29.8	14.4
------	------	------	------	------	------	------

Anh/chị áp dụng thuật toán tìm kiếm gì để đạt hiệu quả tìm kiếm tốt?

Tìm kiếm nhị phân (Binary Search)

Lý do:

Binary Search chỉ áp dụng khi dữ liệu đã sắp xếp (tăng hoặc giảm đều được).

Mỗi lần tìm sẽ loại bỏ một nửa số phần tử.

Độ phức tạp thời gian: $O(\log n) \rightarrow$ rất nhanh với dãy lớn.

Câu 3: Cho cấu trúc Node và cấu trúc cây trong cây nhị phân tìm kiếm như sau:

```
struct Node
{
    float data;
    Node *pLeft;
    Node *pRight;
};
```

```
struct Tree
{
    Node *pRoot;
};
```

Điều kiện để nhận biết một Node có hai nhánh con là gì?

Điều kiện: $\text{node} \rightarrow \text{pLeft} \neq \text{NULL} \ \&\& \ \text{node} \rightarrow \text{pRight} \neq \text{NULL}$

$\text{pLeft} \neq \text{NULL} \rightarrow$ Node có nhánh con trái

$\text{pRight} \neq \text{NULL} \rightarrow$ Node có nhánh con phải

Cả hai cùng khác NULL $\rightarrow$ Node có đủ 2 nhánh con

Chú ý: Từ Câu 4 đến Câu 7 là phần đọc hiểu chương trình rồi ghi kết quả để kiểm tra kỹ năng đọc hiểu mã nguồn (source code). Vì vậy, mã nguồn mô phỏng bên dưới có thể không áp dụng để xây dựng hoàn thiện các cấu trúc dữ liệu. Anh/chị đọc kỹ để ghi kết quả.

Câu 4: Cho biết kết quả của chương trình sau:

```
#include <iostream>
using namespace std;
struct Node
{
    double data;
    Node *pNext = NULL;
};

struct Stack
{
    Node *pTop = NULL;
};

int main()
{
    Stack s;
    s.pTop = new Node({0.4});
    s.pTop->pNext = new Node({1.2});
    s.pTop->data = 3.6;
    cout << s.pTop->pNext->data;
    return 0;
}
```

1.2

Câu 5: Cho biết kết quả của chương trình sau:

```
#include <iostream>
using namespace std;
struct Node
{
    float data;
    Node *pNext = NULL;
    Node *pPre = NULL;
};

struct List
{
    Node *pHead = NULL, *pTail = NULL;
};

int main()
{
    List L;
    L.pHead = new Node({1.2, NULL});
    L.pTail = new Node({2.4, NULL});
    L.pTail->pNext = new Node({4.6, NULL});
    cout << L.pHead->pPre;
    return 0;
}
```

0

Câu 6: Cho biết kết quả của chương trình sau:

```
#include <iostream>
using namespace std;
struct Node
{
    int data;
    Node *pLeft = NULL, *pRight = NULL;
};

struct Tree
{
    Node *pRoot = NULL;
};

int main()
{
    Tree t;
    Node *p;
    p = t.pRoot = new Node({12});
    t.pRoot->pLeft = new Node({29});
    t.pRoot->pRight = new Node({18});
    int *a = new int(39);
    t.pRoot->data = *a;
    cout << p->data;
    return 0;
}
```

39

Câu 7: Cho biết kết quả của chương trình sau:

```
#include <iostream>
using namespace std;
struct Node
{
    float data;
    Node *next;
};

struct Queue
{
    Node *pFront = NULL, *pRear = NULL;
};

int main()
{
    Node *p = new Node({1.6, NULL});
    Queue q;
    q.pRear = q.pFront = p;
    q.pFront->data = 3.1;
    q.pRear->data = 4.2;
    p->data = 5.3;
    cout << q.pRear->data;
    return 0;
}
```

5.3

Câu 8: Cho dãy giá trị sau:

94	69	58	41	29	14
----	----	----	----	----	----

Anh/chị mô phỏng chạy từng bước thuật toán tìm kiếm nhị phân. Áp dụng giá trị cần tìm kiếm là 19 (Chú ý: Không viết code).

Vì mảng giảm dần, nên x nằm bên phải

Cập nhật:

$left = mid + 1 = 3$

Bước 2:

$left = 3, right = 5$

$mid = (3 + 5) / 2 = 4$

$a[mid] = 29$

So sánh:

$19 < 29$

Mảng giảm dần $\rightarrow$ tiếp tục tìm bên phải

Cập nhật:

$left = mid + 1 = 5$

Bước 3:

$left = 5, right = 5$

$mid = (5 + 5) / 2 = 5$

$a[mid] = 14$

So sánh:

$19 > 14$

Mảng giảm dần $\rightarrow$ tìm bên trái

Cập nhật:

$right = mid - 1 = 4$

Kết thúc:

$left = 5, right = 4 \rightarrow left > right$

Dừng thuật toán

Kết luận:

Không tìm thấy giá trị 19 trong dãy

Thuật toán kết thúc sau 3 lần so sánh

Câu 9: Anh/chị mô phỏng chạy từng bước sắp xếp giảm dần bằng thuật toán chèn trực tiếp với bộ dữ liệu sau (Chú ý: Không viết code).

11	69	42	54	83	76	92
----	----	----	----	----	----	----

Quy ước:

Phần bên trái đang được sắp xếp giảm dần
Mỗi bước lấy 1 phần tử chèn vào đúng vị trí

Mô phỏng từng bước:

Bước 1 ($i = 1$, chèn 69)

Dãy đã sắp xếp: [11]

So sánh: $69 > 11 \rightarrow$ chèn trước

69 | 11 | 42 | 54 | 83 | 76 | 92

Bước 2 ($i = 2$, chèn 42)

Dãy đã sắp xếp: [69 | 11]

$42 < 69 \rightarrow$ không chèn trước 69

$42 > 11 \rightarrow$ chèn giữa

69 | 42 | 11 | 54 | 83 | 76 | 92

Bước 3 ($i = 3$, chèn 54)

Dãy đã sắp xếp: [69 | 42 | 11]

$54 < 69$

$54 > 42 \rightarrow$ chèn giữa

69 | 54 | 42 | 11 | 83 | 76 | 92

Bước 4 ($i = 4$, chèn 83)

Dãy đã sắp xếp: [69 | 54 | 42 | 11]

$83 > 69 \rightarrow$ chèn đầu

83 | 69 | 54 | 42 | 11 | 76 | 92

Bước 5 ($i = 5$, chèn 76)

Dãy đã sắp xếp: [83 | 69 | 54 | 42 | 11]

$76 < 83$

$76 > 69 \rightarrow$ chèn sau 83

83 | 76 | 69 | 54 | 42 | 11 | 92

Bước 6 ($i = 6$, chèn 92)

Dãy đã sắp xếp: [83 | 76 | 69 | 54 | 42 | 11]

$92 > 83 \rightarrow$ chèn đầu

92 | 83 | 76 | 69 | 54 | 42 | 11

Kết quả cuối cùng (giảm dần):

92 | 83 | 76 | 69 | 54 | 42 | 11

Câu 10: Cho cấu trúc Node và cấu trúc cây trong cây nhị phân tìm kiếm như sau:

```
struct Node
{
    float data;
    Node *pLeft;
    Node *pRight;
};
```

```
struct Tree
{
    Node *pRoot;
};
```

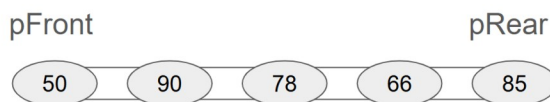
Anh/chị viết hàm (không dùng đệ quy) đếm số lượng Node lá trong cây nhị phân tìm kiếm? Chú ý: Phân tích input và output trước khi viết hàm.

Câu 11: Cho cấu trúc Node và cấu trúc hàng đợi (Queue) như sau:

```
struct Node
{
    int data;
    Node *pPre;
    Node *pNext;
};
```

```
struct Queue
{
    Node *pFront;
    Node *pRear;
};
```

Cấu trúc dữ liệu hàng đợi này được xây dựng dựa trên ý tưởng từ danh sách liên kết đôi. Hình mô phỏng hàng đợi như sau:



Giả sử:

Index dương:	0	1	2	3	4
Index âm:	-5	-4	-3	-2	-1

Giả sử có một bộ chỉ số nguyên (index) âm, dương được dùng để xác định vị trí các Node trong Queue (xem hình minh họa phía trên).

Anh/chị viết hàm Node* getAt(Queue, int) cho phép truyền vào một Queue và một index (âm hoặc dương). Hàm sẽ trả về địa chỉ tương ứng với vị trí của index. Ví dụ gọi hàm:

- getAt(q, -3) => Kết quả trả về địa chỉ Node chứa giá trị 78
- getAt(q, 4) => Kết quả trả về địa chỉ Node chứa giá trị 85
- getAt(q, 9) => Kết quả trả về nullptr (hoặc NULL)

```
Node* getAt(Queue ds, int index)
{
    .....
}
```

PHẦN 2: THỰC HÀNH

Cho cấu trúc Node và cấu trúc danh sách liên kết đơn như sau:


```
struct Node
{
    float data;
    Node *pNext;
};
```

```
struct List
{
    Node *pHead;
    Node *pTail;
};
```

Viết các hàm theo yêu cầu sau:

Câu 1. Viết hàm thêm giá trị vào danh sách liên kết đơn.

Câu 2. Viết hàm đếm các node có giá trị âm.

Câu 3. Viết hàm sắp xếp tăng dần theo thuật toán chọn trực tiếp?

Câu 4. Trong hàm main, khai báo danh sách liên kết đơn và khởi tạo giá trị sau cho danh sách liên kết đơn:

-5.8 3.5 -1.5 -9.4 6.9

Tiếp theo, gọi các hàm câu 1, 2, 3 để kiểm thử.

Chú ý: Không cần khai báo thư viện. Tập trung viết hàm con câu 1, 2, 3 và hàm main.